HealthMate Technical Manual

Version: 1.0
Last Updated: February 8, 2026
Author: Manus AI
Document Type: Technical Reference

Table of Contents

1. System Overview
2. Architecture
3. Technology Stack
4. Development Environment Setup
5. Project Structure
6. Core Services
7. API Reference
8. Database Schema
9. Authentication & Security
10. Integration Guides
11. Deployment
12. Testing
13. Troubleshooting
14. Performance Optimization
15. Compliance & Regulations

---

1. System Overview

1.1 Purpose

HealthMate is a comprehensive mobile health application designed specifically for older adults (ages 50+) and their caregivers. The platform provides AI-powered health assistance, medication management, vital signs monitoring, and family care coordination through a unified, accessible interface.

1.2 Key Capabilities

The system provides four primary capability domains:

Health Monitoring & Tracking
The application enables continuous monitoring of vital signs including blood pressure, heart rate, oxygen saturation, and weight. Integration with Omron medical devices allows automatic data synchronization via Bluetooth Low Energy (BLE) and WiFi protocols. Real-time anomaly detection algorithms analyze incoming data streams and trigger alerts when readings exceed configurable thresholds.

Medication Management
An automated reminder system schedules notifications based on prescription regimens, delivering both push notifications and voice-synthesized reminders through the device's text-to-speech engine. One-tap confirmation logging creates timestamped adherence records, while missed-dose detection algorithms monitor compliance patterns and escalate alerts to designated caregivers after configurable intervals.

AI Health Assistant
A conversational AI interface powered by large language models provides health information, answers medical questions, and offers personalized guidance. The assistant maintains conversation context, accesses the user's health history, and can escalate concerns to human healthcare providers when appropriate.

Family Care Coordination
The caregiver dashboard provides authorized family members with real-time visibility into the senior's health metrics, medication adherence, and activity levels. Automated notifications alert caregivers to missed medications, abnormal vital signs, or achieved health milestones, enabling proactive intervention and celebration of progress.

1.3 Platform Support

The application runs on multiple platforms through a unified codebase:

Platform	Support Level	Distribution Method
iOS (13.4+)	Full native support	App Store / TestFlight
Android (6.0+)	Full native support	Google Play Store
Web (Modern browsers)	Full responsive support	Progressive Web App (PWA)
Windows Desktop	Via web browser	PWA installation
macOS Desktop	Via web browser	PWA installation

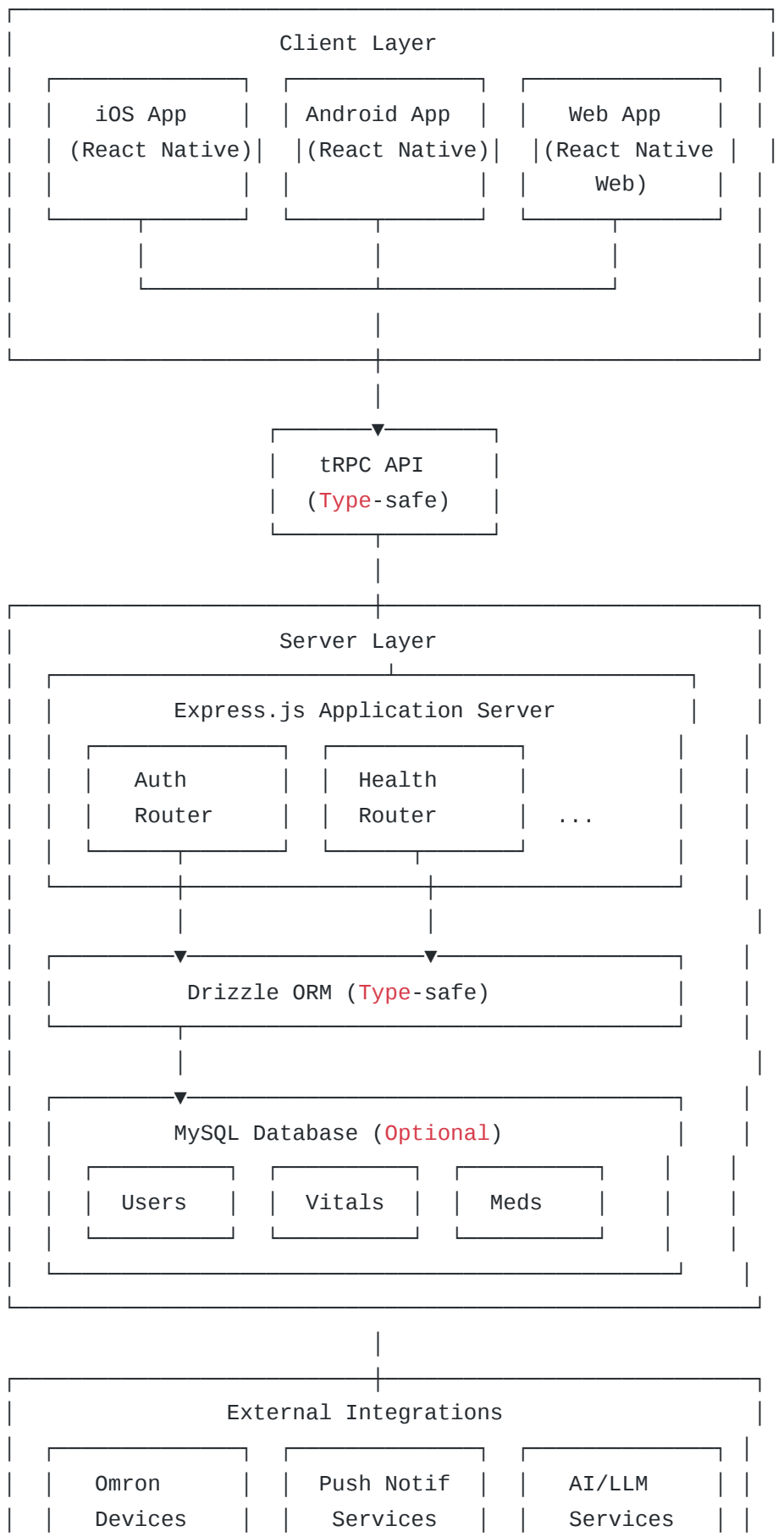
---

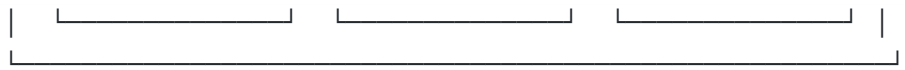
2. Architecture

2.1 System Architecture

HealthMate implements a hybrid architecture combining local-first data storage with optional cloud synchronization. This design ensures the application remains functional without network connectivity while enabling data backup and multi-device access when online.

Client-Server Architecture Diagram:


Client Layer
iOS App
(React Native)
Android App
(React Native)
Web App
(React Native
Web)
tRPC API
(Type-safe)
Server Layer
Express.js Application Server
Auth
Router
Health
Router
...
Drizzle ORM (Type-safe)
MySQL Database (Optional)
Users
Vitals
Meds
External Integrations
Omron
Devices
Push Notif
Services
AI/LLM
Services



2.2 Data Flow Architecture

The application implements a local-first architecture where all user data is stored on-device by default using AsyncStorage (React Native) or IndexedDB (web). This ensures privacy, offline functionality, and instant responsiveness.

Data Persistence Strategy:

Data Type	Primary Storage	Backup Storage	Sync Frequency
User preferences	AsyncStorage	Cloud (optional)	On change
Medication schedules	AsyncStorage	Cloud (optional)	Daily
Vital signs readings	AsyncStorage	Cloud (optional)	Real-time
Health records	AsyncStorage	Cloud (optional)	On change
AI conversation history	AsyncStorage	Cloud (optional)	Per session
Device pairing data	SecureStore	Cloud (encrypted)	On change

2.3 Component Architecture

The application follows a modular component architecture organized into three primary layers:

Presentation Layer (UI Components)
Reusable UI components built with React Native and styled using NativeWind (Tailwind CSS for React Native). All components support both light and dark themes through the ThemeProvider context. The ScreenContainer component ensures proper safe area handling across different device form factors, while ActionCard components provide consistent interaction patterns throughout the application.

Business Logic Layer (Services)
Service modules encapsulate domain-specific logic and maintain separation of concerns. The OmronService handles Bluetooth and WiFi device connectivity, the MedicationReminderService manages scheduling and notifications, and the

MilestoneService tracks achievements and triggers celebrations. Each service operates independently and communicates through well-defined interfaces.

Data Access Layer (Storage & API)
The data layer abstracts storage mechanisms, allowing seamless switching between local AsyncStorage and cloud-based persistence. The tRPC client provides type-safe API communication with automatic request/response validation. Drizzle ORM handles database operations when cloud sync is enabled, ensuring type safety and preventing SQL injection vulnerabilities.

---

3. Technology Stack

3.1 Frontend Technologies

Core Framework:
The application is built on Expo SDK 54, which provides a managed React Native development environment with extensive native module support. React 19 serves as the UI library, offering concurrent rendering and automatic batching for improved performance. TypeScript 5.9 ensures type safety across the entire codebase, catching errors at compile time rather than runtime.

Routing & Navigation:
Expo Router 6 implements file-based routing, where the directory structure in the `app/` folder automatically generates navigation routes. This approach simplifies navigation logic and ensures type-safe route parameters. React Navigation powers the underlying navigation stack, providing native-feeling transitions and gesture handling.

Styling System:
NativeWind 4 brings Tailwind CSS utility classes to React Native, enabling rapid UI development with a consistent design system. The theme configuration in `theme.config.js` defines color palettes that automatically adapt to light and dark modes. Custom CSS variables enable runtime theme switching without component re-renders.

State Management:
The application uses React Context and hooks for global state management, avoiding the complexity of external state libraries. TanStack Query (React Query) handles server

state, providing automatic caching, background refetching, and optimistic updates. Local component state uses useState and useReducer hooks following React best practices.

Animation Library:
React Native Reanimated 4 provides high-performance animations that run on the native thread, ensuring smooth 60fps animations even during JavaScript thread congestion. The library powers the milestone celebration confetti effects, vital signs pulse animations, and screen transitions.

3.2 Backend Technologies

Server Framework:
Express.js serves as the HTTP server framework, providing middleware support for authentication, request validation, and error handling. The server runs on Node.js 22, leveraging modern JavaScript features and improved performance characteristics.

API Layer:
tRPC 11 implements end-to-end type safety between client and server without code generation. TypeScript types defined on the server automatically propagate to the client, eliminating API contract mismatches. The library uses JSON-RPC 2.0 protocol with automatic batching to reduce network requests.

Database:
MySQL serves as the relational database when cloud sync is enabled. Drizzle ORM provides type-safe query building with zero runtime overhead. The ORM generates SQL queries at build time, ensuring optimal performance while maintaining developer ergonomics.

Authentication:
JSON Web Tokens (JWT) handle user authentication, with tokens signed using the jose library. Expo SecureStore encrypts and stores refresh tokens on the device, while access tokens remain in memory. OAuth 2.0 integration enables sign-in with Apple and Google.

3.3 Native Modules

The application leverages numerous Expo SDK modules for native functionality:

Module	Purpose	Platform Support
expo-audio	Voice reminders, audio playback	iOS, Android
expo-notifications	Push notifications, local reminders	iOS, Android, Web
expo-speech	Text-to-speech for medication reminders	iOS, Android
expo-camera	Barcode scanning, pill identification	iOS, Android
expo-bluetooth	Omron device connectivity	iOS, Android
expo-secure-store	Encrypted credential storage	iOS, Android
expo-haptics	Tactile feedback	iOS, Android
expo-keep-awake	Prevent screen sleep during monitoring	iOS, Android, Web
expo-video	Telemedicine video calls	iOS, Android, Web
expo-location	Fall detection, activity tracking	iOS, Android

3.4 Development Tools

Package Manager:
pnpm serves as the package manager, providing faster installation times and more efficient disk space usage through content-addressable storage. The workspace feature enables monorepo management if the project expands to multiple packages.

Build System:
Metro bundler handles JavaScript bundling with support for React Native's platform-specific extensions (.ios.tsx, .android.tsx, .web.tsx). EAS Build compiles native binaries for iOS and Android in the cloud, eliminating the need for local Xcode or Android Studio installations.

Testing Framework:
Vitest provides unit testing with native TypeScript support and compatibility with Jest's API. The framework runs tests in parallel using worker threads, significantly reducing test suite execution time.

Code Quality:
ESLint enforces code style and catches common errors, while Prettier handles

automatic code formatting. TypeScript's strict mode enables comprehensive type checking, including null safety and strict function types.

---

4. Development Environment Setup

4.1 Prerequisites

Before beginning development, ensure the following software is installed on your development machine:

Required Software:

1. **Node.js (v18.0.0 or higher):** Download from nodejs.org and verify installation with `node --version`
2. **pnpm (v9.0.0 or higher):** Install globally using `npm install -g pnpm`
3. **Git (v2.30.0 or higher):** Required for version control and dependency installation
4. **Code Editor:** Visual Studio Code recommended with the following extensions:
    - ESLint
    - Prettier
    - TypeScript and JavaScript Language Features
    - Expo Tools

Platform-Specific Requirements:

For iOS development, macOS is required with:

- Xcode 15.0 or higher
- CocoaPods (installed via `sudo gem install cocoapods`)
- iOS Simulator

For Android development, any operating system with:

- Android Studio (latest stable version)
- Android SDK Platform 33 or higher

- Android Emulator or physical device with USB debugging enabled

4.2 Installation Steps

Step 1: Clone Repository

```
git clone <repository-url>
cd health_companion
```

Step 2: Install Dependencies

```
pnpm install
```

This command installs all required npm packages defined in package.json, including React Native, Expo modules, and development tools. The installation process may take 5-10 minutes depending on network speed.

Step 3: Configure Environment Variables

Create a `.env` file in the project root:

```
 # Database connection (optional - only needed for cloud sync)
DATABASE_URL=mysql://username:password@localhost:3306/healthmate

# API Keys (optional - only if using external services)
OPENAI_API_KEY=sk-...

# Server configuration
PORT=3000
NODE_ENV=development
```

Step 4: Initialize Database (Optional)

If using cloud sync features, push the database schema:

```
pnpm db:push
```

This command uses Drizzle Kit to generate and execute SQL migrations based on the schema definitions in `server/db/schema.ts`.

Step 5: Start Development Server

```
pnpm dev
```

This command starts both the backend API server (port 3000) and the Expo Metro bundler (port 8081). The terminal displays a QR code for testing on physical devices via Expo Go.

4.3 Verification

After starting the development server, verify the installation:

1. **Web Access:** Open http://localhost:8081 in a browser - you should see the HealthMate home screen
2. **Mobile Access:** Scan the QR code with Expo Go app - the app should load on your device
3. **API Health Check:** Visit http://localhost:3000/health - should return `{"status":"ok"}`
4. **TypeScript Compilation:** Run `pnpm check` - should complete without errors

---

5. Project Structure

5.1 Directory Organization

The project follows Expo Router's file-based routing convention, where the directory structure directly maps to application routes:

```
health_companion/
├── app/                          # Application routes and screens
│   ├── (tabs)/                   # Tab-based navigation group
│   │   ├── _layout.tsx           # Tab bar configuration
│   │   └── index.tsx             # Home screen (route: /)
│   ├── _layout.tsx               # Root layout with providers
│   ├── assistant.tsx             # AI assistant (route: /assistant)
│   ├── health.tsx                # Health tracking (route: /health)
│   ├── omron-devices.tsx         # Device management (route: /omron-devices)
│   ├── realtime-vitals.tsx       # Live monitoring (route: /realtime-vitals)
│   ├── milestones.tsx            # Achievements (route: /milestones)
│   └── [30+ other screens]
│
├── services/                     # Business logic services
│   ├── omronService.ts           # Bluetooth/WiFi device integration
│   ├── medicationReminderService.ts  # Medication scheduling
│   ├── milestoneService.ts       # Achievement tracking
│   └── [other services]
│
├── components/                   # Reusable UI components
│   ├── screen-container.tsx      # Safe area wrapper
│   ├── action-card.tsx           # Quick action button
│   ├── themed-view.tsx           # Theme-aware view
│   └── ui/                       # Primitive components
│       └── icon-symbol.tsx       # Icon mapping
│
├── hooks/                        # Custom React hooks
│   ├── use-auth.ts               # Authentication state
│   ├── use-colors.ts             # Theme colors
│   └── use-color-scheme.ts       # Dark mode detection
│
├── lib/                          # Utilities and configuration
│   ├── trpc.ts                   # API client setup
│   ├── utils.ts                  # Helper functions
│   └── theme-provider.tsx        # Theme context
│
├── constants/                    # Application constants
│   └── theme.ts                  # Color palette
│
├── server/                       # Backend API (optional)
│   ├── _core/                    # Server initialization
│   │   └── index.ts              # Express app setup
│   ├── routers/                  # tRPC route handlers
│   │   ├── auth.ts               # Authentication endpoints
│   │   ├── health.ts             # Health data endpoints
```

```
│   │   └── [other routers]
│   └── db/                        # Database layer
│       ├── schema.ts              # Drizzle schema definitions
│       └── index.ts               # Database connection
│
├── assets/                         # Static assets
│   └── images/                    # Icons, splash screens
│
├── app.config.ts                  # Expo configuration
├── tailwind.config.js             # Tailwind CSS config
├── theme.config.js                # Theme tokens
├── package.json                   # Dependencies
├── tsconfig.json                  # TypeScript config
└── todo.md                        # Feature tracking
```

5.2 Routing System

Expo Router automatically generates routes based on file names in the `app/` directory:

File Path	Generated Route	Description
`app/(tabs)/index.tsx`	`/`	Home screen (default tab)
`app/assistant.tsx`	`/assistant`	AI health assistant
`app/health.tsx`	`/health`	Health tracking screen
`app/omron-devices.tsx`	`/omron-devices`	Device management
`app/realtime-vitals.tsx`	`/realtime-vitals`	Live vitals monitor
`app/milestones.tsx`	`/milestones`	Achievement celebrations
`app/[id].tsx`	`/123`	Dynamic route with parameter

Route Groups:
Folders wrapped in parentheses like `(tabs)` create route groups without adding path segments. This allows organizing related screens while keeping URLs clean.

Layout Files:
Files named `_layout.tsx` define shared UI that wraps child routes. The root

`app/_layout.tsx` provides global providers (theme, authentication), while `app/(tabs)/_layout.tsx` configures the tab bar.

5.3 Key Files Explained

app.config.ts:
Expo configuration file defining app metadata, permissions, splash screens, and build settings. This file is processed at build time to generate platform-specific configurations (Info.plist for iOS, AndroidManifest.xml for Android).

theme.config.js:
Central theme configuration defining color palettes for light and dark modes. These tokens are consumed by both Tailwind CSS (compile-time) and the useColors hook (runtime), ensuring consistency across styling approaches.

tailwind.config.js:
Tailwind CSS configuration extending the default theme with custom colors from theme.config.js. NativeWind processes this file to generate React Native styles from Tailwind utility classes.

package.json:
Defines project dependencies, scripts, and metadata. The `scripts` section includes commands for development (`dev`), testing (`test`), building (`build`), and database management (`db:push`).

---

6. Core Services

6.1 Omron Device Service

The OmronService manages Bluetooth Low Energy (BLE) and WiFi connectivity with Omron medical devices, enabling automatic synchronization of blood pressure, weight, and other health measurements.

Service Architecture:

```
// services/omronService.ts

class OmronService {
  private devices: OmronDevice[] = [];
  private scanSubscription: Subscription | null = null;

  // Device discovery via Bluetooth
  async startDeviceScan(): Promise<void>

  // Pair and connect to device
  async pairDevice(deviceId: string): Promise<void>

  // Sync data from connected device
  async syncDeviceData(deviceId: string): Promise<VitalReading[]>

  // Monitor connection status
  async monitorConnection(deviceId: string): Promise<void>
}
```

Bluetooth Protocol:
The service uses the Generic Attribute Profile (GATT) to communicate with Omron devices. Device discovery scans for devices advertising the Omron service UUID (`0x1810` for Blood Pressure Service). Once connected, the service subscribes to characteristic notifications for real-time data updates.

Data Synchronization Flow:

1. User initiates device pairing from the Omron Devices screen
2. Service scans for nearby Bluetooth devices advertising health service UUIDs
3. User selects device from discovered list
4. Service establishes GATT connection and requests pairing
5. Device prompts user to confirm pairing code
6. Service subscribes to measurement characteristics
7. When device takes a reading, it notifies the service via characteristic update
8. Service parses the binary data according to Omron's protocol specification
9. Parsed data is stored locally in AsyncStorage
10. If cloud sync is enabled, data is uploaded to server via tRPC API

WiFi Connectivity:
For Omron devices with WiFi capability, the service configures the device to connect to the user's home network. The device then pushes measurements directly to the HealthMate server via HTTPS. The service polls the server API periodically to retrieve new measurements.

6.2 Medication Reminder Service

The MedicationReminderService implements automated medication scheduling, voice reminders, and adherence tracking with caregiver notifications.

Service Architecture:

```
// services/medicationReminderService.ts

class MedicationReminderService {
  private reminders: MedicationReminder[] = [];

  // Create scheduled reminder
  async createReminder(
    medicationId: string,
    time: string,
    voiceEnabled: boolean
  ): Promise<MedicationReminder>

  // Handle notification received
  async handleNotificationReceived(
    notification: Notification
  ): Promise<void>

  // Log medication taken
  async logMedicationTaken(
    reminderId: string
  ): Promise<void>

  // Check for missed doses
  private async checkMissedDoses(): Promise<void>
}
```

Notification Scheduling:
The service uses Expo Notifications to schedule daily recurring notifications. Each

reminder creates a CalendarTriggerInput with specific hour and minute values. The notification payload includes medication details and a flag indicating whether voice reminders are enabled.

Voice Reminder Implementation:
When a notification is received and voice reminders are enabled, the service uses Expo Speech to synthesize an audio reminder. The text-to-speech message includes the medication name and dosage, spoken at a slower rate (0.9x) for clarity. Voice reminders are disabled on web platforms where speech synthesis has limited browser support.

Adherence Tracking Algorithm:

```
private getLogStatus(scheduledTime: string, takenTime: string):
  'taken' | 'late' | 'missed' {
  const diffMinutes = (takenDate - scheduledDate) / (1000 * 60);

  if (diffMinutes <= 30) return 'taken';      // On time
  if (diffMinutes <= 120) return 'late';      // Within 2 hours
  return 'missed';                             // More than 2 hours late
}
```

The service calculates adherence statistics over configurable time periods (default 30 days), counting doses taken on time, taken late, and missed. The adherence rate is calculated as `(takenOnTime + takenLate) / totalDoses * 100`.

Missed Dose Detection:
A background interval checks every 30 minutes for medications that should have been taken but have no logged confirmation. After the first missed dose, the service sends a follow-up notification. After two consecutive missed doses, the service notifies designated caregivers via push notification and adds an entry to the Care Circle feed.

6.3 Milestone Service

The MilestoneService tracks health achievements, triggers animated celebrations, and notifies family members when milestones are reached.

Service Architecture:

```
// services/milestoneService.ts

class MilestoneService {
  private milestones: Milestone[] = [];
  private achievements: Achievement[] = [];

  // Update progress for all milestones
  async updateAllProgress(): Promise<void>

  // Mark milestone as achieved
  private async achieveMilestone(
    milestone: Milestone
  ): Promise<void>

  // Send celebration notification
  private async sendCelebrationNotification(
    milestone: Milestone
  ): Promise<void>

  // Notify family members
  private async notifyFamily(
    milestone: Milestone
  ): Promise<void>
}
```

Milestone Types:

The service tracks seven predefined milestone types:

Milestone	Requirement	Badge
30-Day Medication Master	Perfect adherence for 30 days	🏆
10,000 Steps Champion	10,000 steps in one day	🏅
7-Day Health Tracker	Log vitals for 7 consecutive days	⭐
Blood Pressure Pro	Healthy BP for 30 days	💙
7-Day Exercise Streak	Exercise 7 consecutive days	🔥
Perfect Week	All goals met for 7 days	💫
Health Score Elite	Health score ≥ 90	🎯

Progress Calculation:
The service runs hourly checks to update milestone progress. For medication adherence, it queries the medication logs and counts consecutive days with perfect adherence. For step count, it retrieves today's activity data from AsyncStorage. For vitals logging, it counts unique days with at least one vital sign entry.

Achievement Celebration Flow:

1. Service detects milestone completion during progress update
2. Milestone is marked as achieved with timestamp
3. Achievement record is created and stored
4. Celebration notification is sent to user's device
5. Family members receive notification via push notification
6. Achievement is added to Care Circle feed for family visibility
7. When user opens app, celebration modal displays with confetti animation
8. User dismisses modal, achievement is marked as celebrated

Animation System:
The celebration screen uses React Native Reanimated to create a confetti effect with 50 animated particles. Each particle has randomized horizontal position, color, and fall delay. The badge scales up with a spring animation and gently rotates back and forth using a looping animation sequence.

---

7. API Reference

7.1 tRPC Router Structure

The backend API is organized into domain-specific routers, each handling related functionality:

```
// server/routers/_app.ts

export const appRouter = router({
  auth: authRouter,         // Authentication and user management
  health: healthRouter,     // Vital signs and health records
  medication: medicationRouter,  // Medication tracking
  caregiver: caregiverRouter,    // Family care coordination
  ai: aiRouter,             // AI assistant interactions
});

export type AppRouter = typeof appRouter;
```

7.2 Authentication Endpoints

auth.register

Creates a new user account with email and password.

```
// Request
{
  email: string;
  password: string;
  name: string;
  dateOfBirth: string;
}

// Response
{
  user: {
    id: string;
    email: string;
    name: string;
  };
  accessToken: string;
  refreshToken: string;
}
```

auth.login

Authenticates user and returns JWT tokens.

```
// Request
{
  email: string;
  password: string;
}

// Response
{
  user: User;
  accessToken: string;
  refreshToken: string;
}
```

auth.refreshToken

Exchanges refresh token for new access token.

```
// Request
{
  refreshToken: string;
}

// Response
{
  accessToken: string;
}
```

7.3 Health Data Endpoints

health.recordVitals

Stores a new vital signs reading.

```
// Request
{
  type: 'blood_pressure' | 'heart_rate' | 'oxygen_saturation' | 'weight';
  timestamp: string;
  systolic?: number;
  diastolic?: number;
  heartRate?: number;
  oxygenSaturation?: number;
  weight?: number;
  source: 'manual' | 'omron' | 'wearable';
}

// Response
{
  id: string;
  timestamp: string;
  status: 'normal' | 'warning' | 'critical';
}
```

health.getVitalsHistory

Retrieves vital signs history with optional filtering.

```
// Request
{
  type?: 'blood_pressure' | 'heart_rate' | 'oxygen_saturation' | 'weight';
  startDate?: string;
  endDate?: string;
  limit?: number;
}

// Response
{
  readings: VitalReading[];
  summary: {
    average: number;
    min: number;
    max: number;
    trend: 'improving' | 'stable' | 'declining';
  };
}
```

7.4 Medication Endpoints

medication.createReminder

Schedules a new medication reminder.

```
// Request
{
  medicationId: string;
  medicationName: string;
  dosage: string;
  time: string;  // HH:MM format
  frequency: 'daily' | 'weekly' | 'as_needed';
  voiceEnabled: boolean;
}

// Response
{
  reminderId: string;
  nextReminder: string;
}
```

medication.logTaken

Records medication as taken.

```
// Request
{
  reminderId: string;
  takenAt?: string;  // Defaults to now
}

// Response
{
  logId: string;
  adherenceRate: number;
  streak: number;
}
```

7.5 AI Assistant Endpoints

ai.sendMessage

Sends message to AI assistant and receives response.

```
// Request
{
  message: string;
  conversationId?: string;
  context?: {
    recentVitals?: VitalReading[];
    medications?: Medication[];
  };
}

// Response
{
  response: string;
  conversationId: string;
  suggestions?: string[];
}
```

7.6 Error Handling

All API endpoints return standardized error responses:

```
{
  error: {
    code: string;  // e.g., 'UNAUTHORIZED', 'VALIDATION_ERROR'
    message: string;
    details?: any;
  }
}
```

Common error codes:

Code	HTTP Status	Description
UNAUTHORIZED	401	Invalid or expired authentication token
FORBIDDEN	403	User lacks permission for this action
NOT_FOUND	404	Requested resource does not exist
VALIDATION_ERROR	400	Request data failed validation
INTERNAL_SERVER_ERROR	500	Unexpected server error

---

8. Database Schema

8.1 Schema Overview

The database uses MySQL with Drizzle ORM for type-safe queries. The schema is defined in `server/db/schema.ts` using Drizzle's schema builder.

Core Tables:

```
// Users table
export const users = mysqlTable('users', {
  id: varchar('id', { length: 36 }).primaryKey(),
  email: varchar('email', { length: 255 }).unique().notNull(),
  passwordHash: varchar('password_hash', { length: 255 }).notNull(),
  name: varchar('name', { length: 255 }).notNull(),
  dateOfBirth: date('date_of_birth'),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow().onUpdateNow(),
});

// Vital signs readings
export const vitals = mysqlTable('vitals', {
  id: varchar('id', { length: 36 }).primaryKey(),
  userId: varchar('user_id', { length: 36 }).references(() => users.id),
  type: varchar('type', { length: 50 }).notNull(),
  timestamp: timestamp('timestamp').notNull(),
  systolic: int('systolic'),
  diastolic: int('diastolic'),
  heartRate: int('heart_rate'),
  oxygenSaturation: int('oxygen_saturation'),
  weight: decimal('weight', { precision: 5, scale: 2 }),
  source: varchar('source', { length: 50 }),
  deviceId: varchar('device_id', { length: 255 }),
});

// Medications
export const medications = mysqlTable('medications', {
  id: varchar('id', { length: 36 }).primaryKey(),
  userId: varchar('user_id', { length: 36 }).references(() => users.id),
  name: varchar('name', { length: 255 }).notNull(),
  dosage: varchar('dosage', { length: 100 }),
  frequency: varchar('frequency', { length: 50 }),
  startDate: date('start_date'),
  endDate: date('end_date'),
  active: boolean('active').default(true),
});

// Medication logs
export const medicationLogs = mysqlTable('medication_logs', {
  id: varchar('id', { length: 36 }).primaryKey(),
  medicationId: varchar('medication_id', { length: 36 })
    .references(() => medications.id),
  scheduledTime: timestamp('scheduled_time').notNull(),
  takenTime: timestamp('taken_time'),
```

```
  status: varchar('status', { length: 20 }),
});

// Caregivers
export const caregivers = mysqlTable('caregivers', {
  id: varchar('id', { length: 36 }).primaryKey(),
  seniorId: varchar('senior_id', { length: 36 })
    .references(() => users.id),
  caregiverId: varchar('caregiver_id', { length: 36 })
    .references(() => users.id),
  relationship: varchar('relationship', { length: 50 }),
  permissions: json('permissions'),
  createdAt: timestamp('created_at').defaultNow(),
});
```

8.2 Relationships

User → Vitals (One-to-Many):
Each user can have multiple vital sign readings. The `userId` foreign key in the vitals table references the users table.

User → Medications (One-to-Many):
Each user can have multiple medications. The `userId` foreign key in the medications table references the users table.

Medication → Medication Logs (One-to-Many):
Each medication can have multiple log entries tracking when doses were taken. The `medicationId` foreign key references the medications table.

User → Caregivers (Many-to-Many):
The caregivers table implements a many-to-many relationship between seniors and their caregivers. Both `seniorId` and `caregiverId` reference the users table.

8.3 Indexes

Performance-critical queries use database indexes:

```
export const vitalsUserIdIndex = index('vitals_user_id_idx')
  .on(vitals.userId);

export const vitalsTimestampIndex = index('vitals_timestamp_idx')
  .on(vitals.timestamp);

export const medicationLogsScheduledTimeIndex =
  index('medication_logs_scheduled_time_idx')
  .on(medicationLogs.scheduledTime);
```

These indexes significantly improve query performance for common operations like retrieving a user's vital signs history or finding upcoming medication reminders.

---

9. Authentication & Security

9.1 Authentication Flow

The application implements JWT-based authentication with refresh token rotation:

Registration Flow:

1. User submits email, password, and profile information
2. Server validates email format and password strength (minimum 8 characters, requires uppercase, lowercase, number)
3. Password is hashed using bcrypt with salt rounds of 12
4. User record is created in database
5. Server generates access token (15-minute expiry) and refresh token (7-day expiry)
6. Refresh token is stored in Expo SecureStore (encrypted hardware-backed storage)
7. Access token is kept in memory only (never persisted)

Login Flow:

1. User submits email and password

2. Server retrieves user record by email
3. Password is verified using bcrypt.compare()
4. If valid, server generates new access and refresh tokens
5. Tokens are returned to client
6. Client stores refresh token in SecureStore and access token in memory

Token Refresh Flow:

1. When access token expires (detected via 401 response), client automatically calls refresh endpoint
2. Client sends refresh token from SecureStore
3. Server validates refresh token signature and expiry
4. If valid, server generates new access token and refresh token
5. Old refresh token is invalidated (one-time use)
6. New tokens are returned to client

9.2 Authorization

The application implements role-based access control (RBAC) with three roles:

Role	Permissions	Description
Senior	Full access to own data	Primary user of the application
Caregiver	Read access to linked senior's data	Family member or professional caregiver
Admin	Full system access	System administrators

Permission Checking:
Each tRPC endpoint includes middleware that verifies the user's role and permissions:

```
const protectedProcedure = publicProcedure.use(async ({ ctx, next }) => {
  if (!ctx.user) {
    throw new TRPCError({ code: 'UNAUTHORIZED' });
  }
  return next({ ctx: { ...ctx, user: ctx.user } });
});
```

9.3 Data Encryption

At Rest:
Sensitive data stored in the database is encrypted using AES-256-GCM. Encryption keys are managed through environment variables and never committed to version control. Expo SecureStore uses hardware-backed encryption on iOS (Keychain) and Android (Keystore).

In Transit:
All API communication uses HTTPS with TLS 1.3. The server enforces HTTPS and rejects plaintext HTTP connections in production. Certificate pinning can be enabled for additional security against man-in-the-middle attacks.

Local Storage:
AsyncStorage data is encrypted on iOS and Android using platform-specific encryption APIs. On web, IndexedDB data is encrypted using the Web Crypto API before storage.

9.4 Security Best Practices

Input Validation:
All user inputs are validated using Zod schemas before processing. This prevents SQL injection, XSS attacks, and other injection vulnerabilities.

Rate Limiting:
API endpoints implement rate limiting to prevent brute force attacks. The default limit is 100 requests per 15 minutes per IP address. Authentication endpoints have stricter limits of 5 attempts per 15 minutes.

CORS Configuration:
Cross-Origin Resource Sharing (CORS) is configured to only allow requests from

authorized domains. In development, localhost is permitted. In production, only the deployed web app domain is allowed.

Dependency Security:
Dependencies are regularly audited using `pnpm audit`. Critical security updates are applied promptly. Automated Dependabot alerts notify of vulnerabilities in dependencies.

---

10. Integration Guides

10.1 Omron Device Integration

Supported Devices:
The application supports Omron blood pressure monitors with Bluetooth connectivity, including models BP7000, BP5450, and Evolv. WiFi-enabled models like the Complete are also supported.

Bluetooth Pairing Process:

```
// 1. Request Bluetooth permissions
const { status } = await Bluetooth.requestPermissionsAsync();

// 2. Start scanning for devices
const devices = await omronService.startDeviceScan();

// 3. User selects device from list
const selectedDevice = devices.find(d => d.id === deviceId);

// 4. Pair with device
await omronService.pairDevice(selectedDevice.id);

// 5. Subscribe to measurements
await omronService.monitorConnection(selectedDevice.id);
```

Data Format:
Omron devices transmit blood pressure data using the Bluetooth Blood Pressure Service (UUID 0x1810). The measurement characteristic (UUID 0x2A35) contains:

- Systolic pressure (mmHg)
- Diastolic pressure (mmHg)
- Mean arterial pressure (mmHg)
- Timestamp
- Pulse rate (bpm)
- User ID
- Measurement status

WiFi Configuration:
For WiFi-enabled devices, the setup process involves:

1. Device creates temporary WiFi access point
2. User connects phone to device's WiFi network
3. App sends home WiFi credentials to device
4. Device connects to home network
5. Device registers with HealthMate server
6. Measurements are automatically uploaded to server

10.2 Push Notification Integration

Expo Push Notification Service:
The application uses Expo's push notification service for reliable delivery across iOS and Android. The service handles platform-specific differences and provides a unified API.

Setup Process:

```
// 1. Request notification permissions
const { status } = await Notifications.requestPermissionsAsync();

// 2. Get Expo push token
const token = await Notifications.getExpoPushTokenAsync({
  projectId: 'your-project-id',
});

// 3. Register token with server
await trpc.user.registerPushToken.mutate({ token: token.data });

// 4. Configure notification handler
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: true,
  }),
});
```

Notification Categories:
The application defines several notification categories with custom actions:

Category	Actions	Use Case
medication	Take, Snooze, Skip	Medication reminders
vital_alert	View, Dismiss	Abnormal vital signs
milestone	Celebrate, Share	Achievement unlocked
caregiver_alert	Call, Message, Dismiss	Senior needs attention

10.3 AI Assistant Integration

The AI assistant uses a large language model API to provide health information and guidance. The integration supports multiple LLM providers.

Configuration:

```
// lib/ai-config.ts
export const aiConfig = {
  provider: 'openai',  // or 'anthropic', 'google'
  model: 'gpt-4',
  temperature: 0.7,
  maxTokens: 500,
  systemPrompt: `You are a helpful health assistant for seniors...`,
};
```

Conversation Flow:

```
// 1. User sends message
const userMessage = "What should I do if I forget to take my medication?";

// 2. Build context with user's health data
const context = {
  recentVitals: await getRecentVitals(),
  medications: await getMedications(),
  age: user.age,
};

// 3. Send to AI with context
const response = await trpc.ai.sendMessage.mutate({
  message: userMessage,
  context,
});

// 4. Display response to user
displayMessage(response.response);
```

Safety Guardrails:
The AI assistant includes several safety measures:

- Disclaimers that advice is not a substitute for professional medical care
- Escalation prompts for emergency situations (chest pain, difficulty breathing)
- Refusal to provide specific medication dosage recommendations
- Encouragement to consult healthcare providers for serious concerns

---

11. Deployment

11.1 Web Deployment

The web version can be deployed to any static hosting provider:

Build Process:

```
# Generate static web build
npx expo export:web

# Output directory: web-build/
```

Hosting Options:

Provider	Setup Complexity	Cost	CDN
Vercel	Low	Free tier available	Yes
Netlify	Low	Free tier available	Yes
AWS S3 + CloudFront	Medium	Pay-as-you-go	Yes
Firebase Hosting	Low	Free tier available	Yes

Environment Configuration:
Create platform-specific environment files:

```
.env.production
.env.staging
.env.development
```

11.2 Mobile App Deployment

iOS App Store:

1. Enroll in Apple Developer Program ($99/year)
2. Create App Store Connect listing

3. Build production IPA using EAS Build:

```
eas build --platform ios --profile production
```

4. Upload to App Store Connect via Transporter
5. Submit for review
6. Typical review time: 1-3 days

Google Play Store:

1. Create Google Play Developer account ($25 one-time fee)
2. Create app listing in Play Console
3. Build production AAB using EAS Build:

```
eas build --platform android --profile production
```

4. Upload AAB to Play Console
5. Complete store listing and compliance forms
6. Submit for review
7. Typical review time: 1-7 days

11.3 Backend Deployment

Server Requirements:

- Node.js 18+ runtime
- MySQL 8.0+ database
- 1GB RAM minimum (2GB recommended)
- HTTPS certificate (Let's Encrypt recommended)

Deployment Options:

Docker Container:

```
 FROM node:18-alpine
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build
CMD ["pnpm", "start"]
```

Platform-as-a-Service:

Provider	Ease of Use	Scaling	Cost
Railway	High	Automatic	Free tier + usage
Render	High	Automatic	Free tier + usage
Heroku	High	Manual/Auto	Paid plans
AWS Elastic Beanstalk	Medium	Automatic	Pay-as-you-go

11.4 Database Migration

Production Migration Process:

```
# 1. Generate migration SQL
pnpm drizzle-kit generate:mysql

# 2. Review generated SQL in drizzle/ directory

# 3. Apply migration to production database
pnpm drizzle-kit push:mysql --config=drizzle.config.ts
```

Backup Strategy:
Implement automated daily backups of the production database. Store backups in geographically separate location. Test restore process monthly.

---

12. Testing

12.1 Unit Testing

The project uses Vitest for unit testing with TypeScript support:

Test Structure:

```
// services/medicationReminderService.test.ts

import { describe, it, expect, beforeEach } from 'vitest';
import { medicationReminderService } from './medicationReminderService';

describe('MedicationReminderService', () => {
  beforeEach(async () => {
    await medicationReminderService.initialize();
  });

  it('should create a medication reminder', async () => {
    const reminder = await medicationReminderService.createReminder(
      'med-123',
      'Aspirin',
      '81mg',
      '09:00',
      true
    );

    expect(reminder.medicationName).toBe('Aspirin');
    expect(reminder.time).toBe('09:00');
    expect(reminder.enabled).toBe(true);
  });

  it('should calculate adherence rate correctly', async () => {
    // Test implementation
  });
});
```

Running Tests:

```
# Run all tests
pnpm test

# Run tests in watch mode
pnpm test --watch

# Run tests with coverage
pnpm test --coverage

# Run specific test file
pnpm test services/medicationReminderService.test.ts
```

12.2 Integration Testing

Integration tests verify that multiple components work together correctly:

```
// tests/integration/medication-flow.test.ts

describe('Medication Reminder Flow', () => {
  it('should send notification and log medication taken', async () => {
    // 1. Create reminder
    const reminder = await medicationReminderService.createReminder(...);

    // 2. Trigger notification
    await medicationReminderService.handleNotificationReceived(...);

    // 3. Log medication taken
    await medicationReminderService.logMedicationTaken(reminder.id);

    // 4. Verify adherence updated
    const stats = await medicationReminderService.getAdherenceStats();
    expect(stats.adherenceRate).toBeGreaterThan(0);
  });
});
```

12.3 End-to-End Testing

E2E tests simulate real user interactions using Detox (React Native) or Playwright (web):

```
// e2e/medication-reminder.e2e.ts

describe('Medication Reminder E2E', () => {
  it('should create reminder and receive notification', async () => {
    await device.launchApp();

    // Navigate to medications screen
    await element(by.text('Log Medication')).tap();

    // Create new reminder
    await element(by.id('add-reminder-button')).tap();
    await element(by.id('medication-name-input')).typeText('Aspirin');
    await element(by.id('time-picker')).tap();
    // ... set time
    await element(by.id('save-button')).tap();

    // Verify reminder appears in list
    await expect(element(by.text('Aspirin'))).toBeVisible();
  });
});
```

---

13. Troubleshooting

13.1 Common Issues

Metro Bundler Fails to Start:

```
# Clear cache and restart
npx expo start -c

# If port 8081 is in use
lsof -ti:8081 | xargs kill
```

TypeScript Errors After Dependency Update:

```
# Regenerate TypeScript declarations
pnpm check

# Clear TypeScript cache
rm -rf node_modules/.cache
```

Bluetooth Connection Fails:

1. Verify Bluetooth permissions in app.json
2. Check device is in pairing mode
3. Ensure device is not connected to another app
4. Restart Bluetooth on phone
5. Check console logs for specific error codes

Push Notifications Not Received:

1. Verify Expo push token is registered
2. Check notification permissions granted
3. Ensure app is not in battery optimization mode (Android)
4. Verify Expo project ID matches in app.config.ts
5. Test with Expo's push notification tool

13.2 Performance Issues

Slow List Scrolling:

- Ensure using FlatList instead of ScrollView with .map()
- Implement getItemLayout for fixed-height items
- Use React.memo() for list item components
- Reduce number of re-renders with useCallback

High Memory Usage:

- Check for memory leaks in useEffect cleanup
- Unsubscribe from event listeners on unmount

- Release audio players and video players when done
- Limit size of AsyncStorage data

Slow API Responses:

- Add database indexes for frequently queried fields
- Implement response caching with React Query
- Use pagination for large data sets
- Enable gzip compression on server

13.3 Debug Tools

React Native Debugger:
Download from GitHub and connect to Metro bundler. Provides Redux DevTools, React DevTools, and network inspector.

Flipper:
Facebook's debugging platform with plugins for network inspection, database browsing, and layout inspection.

Expo Dev Tools:
Access via `npx expo start` and press `m` to open. Provides device logs, performance monitor, and element inspector.

---

14. Performance Optimization

14.1 Frontend Optimization

Code Splitting:
Expo Router automatically code-splits routes, loading screens only when navigated to. For large components, use React.lazy():

```
const HeavyComponent = React.lazy(() => import('./HeavyComponent'));

<Suspense fallback={<LoadingSpinner />}>
  <HeavyComponent />
</Suspense>
```

Image Optimization:
Use Expo Image component with caching and placeholder support:

```
<Image
  source={{ uri: imageUrl }}
  placeholder={blurhash}
  contentFit="cover"
  transition={200}
  cachePolicy="memory-disk"
/>
```

List Virtualization:
Always use FlatList for long lists to render only visible items:

```
<FlatList
  data={items}
  renderItem={({ item }) => <ListItem item={item} />}
  keyExtractor={item => item.id}
  getItemLayout={(data, index) => ({
    length: ITEM_HEIGHT,
    offset: ITEM_HEIGHT * index,
    index,
  })}
  removeClippedSubviews={true}
  maxToRenderPerBatch={10}
  windowSize={5}
/>
```

14.2 Backend Optimization

Database Query Optimization:

```
// Bad: N+1 query problem
const users = await db.select().from(usersTable);
for (const user of users) {
  const vitals = await db.select()
    .from(vitalsTable)
    .where(eq(vitalsTable.userId, user.id));
}

// Good: Single query with join
const usersWithVitals = await db.select()
  .from(usersTable)
  .leftJoin(vitalsTable, eq(usersTable.id, vitalsTable.userId));
```

Response Caching:
Implement Redis caching for frequently accessed data:

```
const cachedData = await redis.get(`vitals:${userId}`);
if (cachedData) {
  return JSON.parse(cachedData);
}

const data = await db.select().from(vitalsTable)
  .where(eq(vitalsTable.userId, userId));

await redis.setex(`vitals:${userId}`, 300, JSON.stringify(data));
return data;
```

Connection Pooling:
Configure database connection pool for optimal performance:

```
const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  connectionLimit: 10,
  queueLimit: 0,
});
```

---

15. Compliance & Regulations

15.1 HIPAA Compliance

If the application handles Protected Health Information (PHI), HIPAA compliance is required:

Technical Safeguards:

- End-to-end encryption for data in transit (TLS 1.3)
- Encryption at rest using AES-256
- Secure authentication with multi-factor authentication option
- Automatic session timeout after 15 minutes of inactivity
- Audit logging of all PHI access

Administrative Safeguards:

- Business Associate Agreements (BAA) with all third-party services
- Regular security risk assessments
- Employee training on HIPAA requirements
- Incident response plan for data breaches

Physical Safeguards:

- Secure data center with restricted access
- Backup and disaster recovery procedures
- Workstation security policies

15.2 GDPR Compliance

For users in the European Union, GDPR compliance is mandatory:

User Rights:

- Right to access: Users can download all their data
- Right to erasure: Users can delete their account and all data
- Right to portability: Data export in machine-readable format (JSON)

- Right to rectification: Users can correct inaccurate data

Implementation:

```
// Data export endpoint
export const exportUserData = async (userId: string) => {
  const userData = await db.select().from(usersTable)
    .where(eq(usersTable.id, userId));
  const vitals = await db.select().from(vitalsTable)
    .where(eq(vitalsTable.userId, userId));
  const medications = await db.select().from(medicationsTable)
    .where(eq(medicationsTable.userId, userId));

  return {
    user: userData[0],
    vitals,
    medications,
    exportDate: new Date().toISOString(),
  };
};

// Data deletion endpoint
export const deleteUserData = async (userId: string) => {
  await db.delete(vitalsTable).where(eq(vitalsTable.userId, userId));
  await db.delete(medicationsTable).where(eq(medicationsTable.userId,
userId));
  await db.delete(usersTable).where(eq(usersTable.id, userId));
};
```

15.3 FDA Regulations

The application is designed as a wellness tool and does not diagnose, treat, or prevent disease. However, if future features include clinical decision support or diagnostic capabilities, FDA clearance may be required.

Current Classification:
The application falls under the FDA's "Enforcement Discretion" category for low-risk general wellness products. No FDA clearance is currently required.

Disclaimers:
The application includes prominent disclaimers:

- “This app is for informational purposes only and is not a substitute for professional medical advice”
- “Always consult your healthcare provider before making medical decisions”
- “In case of emergency, call 911 or your local emergency number”

15.4 Accessibility Compliance

The application complies with WCAG 2.1 Level AA standards:

Implementation:

- All interactive elements have accessible labels
- Color contrast ratios meet 4.5:1 minimum
- Font sizes are scalable and respect system settings
- Screen reader support for all content
- Keyboard navigation support (web version)

```
<TouchableOpacity
  accessible={true}
  accessibilityLabel="Take medication now"
  accessibilityHint="Logs this medication as taken"
  accessibilityRole="button"
>
  <Text>Take Now</Text>
</TouchableOpacity>
```

Appendix A: Glossary

AsyncStorage: React Native’s key-value storage system for persisting data locally on the device.

BLE (Bluetooth Low Energy): Wireless communication protocol optimized for low power consumption, used for connecting medical devices.

Drizzle ORM: TypeScript-first ORM providing type-safe database queries with zero runtime overhead.

EAS (Expo Application Services): Cloud-based build and submission service for React Native apps.

Expo: Framework and platform for building universal React Native applications.

GATT (Generic Attribute Profile): Bluetooth protocol defining how data is organized and exchanged between devices.

JWT (JSON Web Token): Compact, URL-safe token format for securely transmitting information between parties.

Metro: JavaScript bundler for React Native applications.

NativeWind: Tailwind CSS implementation for React Native.

tRPC: End-to-end type-safe RPC framework for TypeScript applications.

Vitest: Fast unit testing framework with native TypeScript support.

Appendix B: References

This technical manual was created based on the HealthMate application codebase and industry best practices for React Native development, healthcare application design, and regulatory compliance.

Document Version: 1.0
Last Updated: February 8, 2026
Prepared by: Manus AI
Contact: For technical support, refer to the project repository or contact the development team.